

Memoria Práctica 1

Aprendizaje Automático

Jorge Alcalde Vesteiro
Andy Barcia Rodríguez
Pedro Pablo López Mena

Índice

1) Introducción	3
2) Pre-procesado de datos	3
2.a) Tratado de missing values, valores nulos y valores incorrectos	4
2.b) Tratamiento de outliers	5
2.c) Normalización	6
2.d) SMOTE	6
3) Reducción de dimensionalidad	7
3.a) <i>Principal Component Analysis</i> (PCA)	8
3.b) <i>Linear Discriminant Analysis</i> (LDA)	9
4) Creación y Validación	9
4.a) Validación Cruzada Anidada	10
4.b) Proceso de Validación Completo	12
5) Resultados	13
5.a) Hiperparámetros elegidos	14
5.b) Tablas	15
5.c) Pruebas en <i>dataset</i> de test	16
6) Bibliografía	16

1) Introducción

Esta práctica consiste en pasar por todo el proceso de análisis inteligente de datos de un conjunto de datos de cáncer de mama. Para ello se pasará por todo el proceso de análisis, desde limpiar los datos (utilizando distintas posibles técnicas), reducir la dimensionalidad de los mismos, y crear y validar los modelos.

En total se utilizarán 4 tipos de modelos distintos de clasificación supervisada:

- **k vecinos más próximos**, un modelo en el que se buscan los k vecinos del *training set* más próximos a los datos de *test*, prediciéndose la clase que sea más común entre los vecinos.

El único parámetro que se tendrá que ajustar es el número de vecinos k , y se ajustará y validará en grupo.

- **Regresión logística**, un modelo en el que se tratan de ajustar los parámetros de una función logística para clasificación binaria.

Los únicos parámetros a ajustar están relacionados con la regularización del modelo (para prevenir *overfitting*), y son el de «penalty» (especifica el tipo de regularización) y C (es la inversa de la fuerza de regularización).

Este modelo será ajustado y validado por Pedro Pablo.

- **Árbol de decisión**, un modelo en el que se utiliza una estructura en forma de árbol para tomar las decisiones de clasificación. Los nodos del árbol de clasificación se dividen de forma dinámica en base a distintos criterios de los datos de entrenamiento.

Los parámetros a ajustar son «max_depth» (la profundidad máxima del árbol de decisión), «min_samples_split» (el número de muestras mínimo necesarios para seguir dividiendo un nodo interno) y «min_samples_leaf» (el número de muestras mínimo necesario en un nodo hoja).

Este modelo será ajustado y validado por Andy.

- **Perceptrón multicapa**, un modelo que utiliza varias capas de neuronas artificiales totalmente interconectadas junto con funciones de transformación no lineales (ej, ReLU) para clasificar unas neuronas de entrada en unas neuronas de salida.

Los parámetros a ajustar serán el número de capas ocultas, y el número de neuronas por capa. Opcionalmente también se podrá ajustar la velocidad de entrenamiento y el número máximo de iteraciones que se entrenará.

Este modelo será ajustado y validado por Jorge.

2) Pre-procesado de datos

El apartado de pre-procesado de datos sirve para tratar con los datos del *dataset* que se va a usar para el entrenamiento, de tal manera que se dejan únicamente datos de alta calidad para entrenar al modelo. El pre-procesamiento de datos puede contar con muchos apartados pero en este caso se realizan las siguientes acciones:

- **Eliminar datos nulos**
- **Reemplazar datos nulos por el promedio del total de datos del mismo tipo**
- **Reemplazar los datos por K-neighbors**

La calidad y cantidad de los datos de entrenamiento es importante a la hora de definir la mejor estrategia a ser utilizada. En un *dataset* con gran cantidad de datos, eliminar información no tendrá un gran

efecto global sobre el entrenamiento posterior de los modelos, sin embargo, en el caso de este trabajo, la cantidad de información es limitada, y por lo mismo es más conveniente utilizar una de las últimas dos opciones de pro-procesado de los datos.

2.a) Tratado de missing values, valores nulos y valores incorrectos

El título del apartado es bastante explicativo. Cuando estamos tratando con un *dataset* de entrenamiento, lo primero que tenemos que tener en cuenta es como actuamos ante los datos vacíos y/o incorrectos. A grandes rasgos, las opciones que tenemos ante estos datos son ignorarlos, borrarlos o rellenarlos.

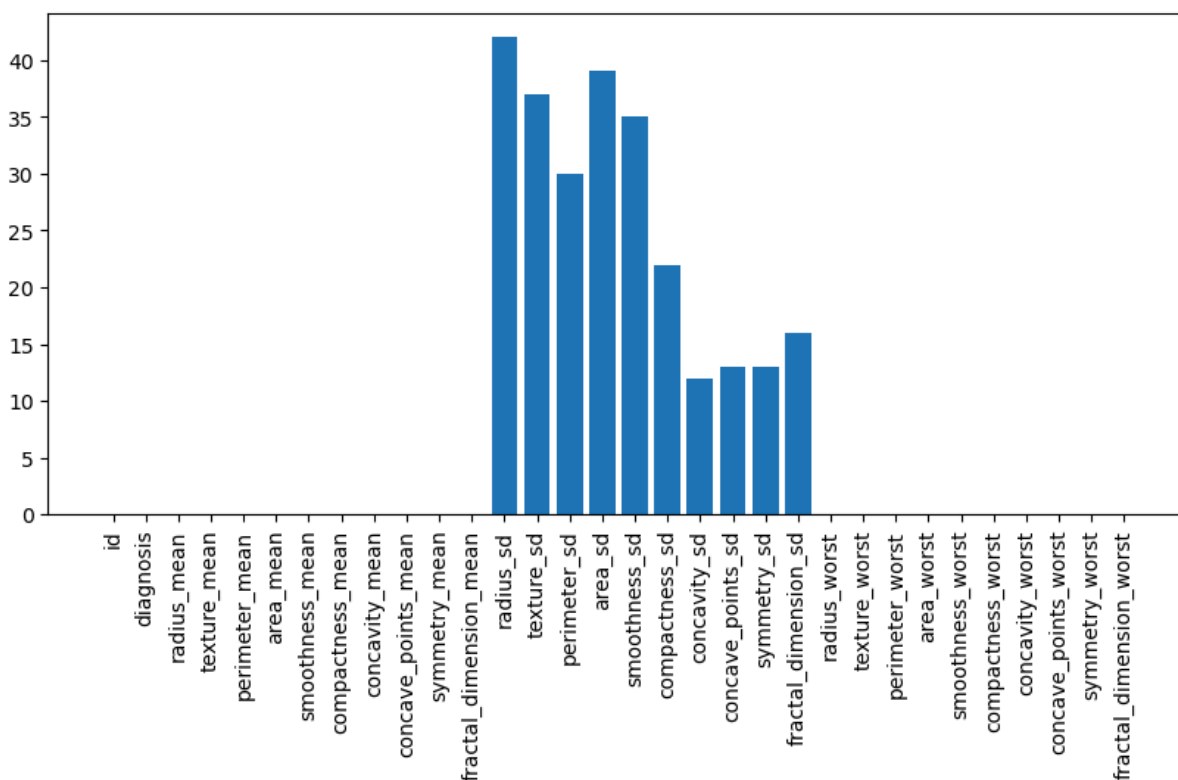


Figura 1: Número de filas que tienen una columna dada vacía.

En este caso nosotros ofrecemos tres maneras de tratarlos:

- **Eliminándolos**

La manera más sencilla de tratar los datos, como se ha mencionado previamente, es eliminando aquellas filas que contengan datos nulos.

```
def eliminar_filas_vacias(df):
    return df.copy().dropna()
```

Aunque nosotros realicemos un borrado de todos los elementos con atributos nulos, en caso de querer aumentar la complejidad del borrado, se podría debatir borrar solamente aquellas filas con N atributos nulos y para aquellas filas con $n < N$ atributos aplicar tratamiento de relleno como los que se ofrecen a continuación.

- **SimpleImputer**

Esta manera de tratamiento de datos entraría dentro de una solución de relleno. El objetivo de esta acción es reallizar un relleno de datos usando algún tipo de herramienta estadística sencilla. Para ello usamos el modulo `sklearn.impute.SimpleImputer`. Este recibe como parametros el *dataset* a rellenar y

el tipo de «estrategia» a emplear. Nosotros decidimos utilizar la media (strategy=mean), sin embargo con este modulo puedes usar también la mediana, la moda y un valor constante que se designe.

```
def rellenar_filas_vacias_con_media(df):
    imp_mean = SimpleImputer(missing_values=np.nan, strategy='mean')
    imp_mean.fit(df)

    df_updated = df.copy()
    vacio = df.isna().any(axis=0) > 0
    df_updated.loc[:, vacio] = imp_mean.transform(df)[:, vacio]

    return df_updated
```

• KNNImputer

KNNImputer es otra estrategia de relleno de datos. En este caso, para realizar dicho relleno lo que se emplea es la estrategia de los vecinos más próximos. La estrategia de vecinos más próximos es fácil de definir en teoría, rellenas los datos con valores de elementos que tienen datos con un alto grado de parecido. De hecho, en sklearn.impute.KNNImputer definen el proceso como «Two samples are close if the features that neither is missing are close». En este caso el valor que modificamos es el del numero de vecinos que se tienen en cuenta para el imput de valor, donde arbitrariamente seleccionamos 15 (default es 5) y el peso de los vecinos, que en este caso dejamos como uniforme.

```
def rellenar_filas_vacias_con_vecinos(df, vecinos=15):
    knn_imputer = KNNImputer(n_neighbors=vecinos, weights="uniform")

    df_updated = df.copy()
    vacio = df.isna().any(axis=0) > 0
    df_updated.loc[:, vacio] = knn_imputer.fit_transform(df)[:, vacio]

    return df_updated
```

Como nota final destacar que los datos generados con ambos imputers no presentan una diferencia destacable y en papel deberían generar soluciones muy similares. Los comparamos de una manera simple y vemos una diferencia entre ellos de menos de un 1%

```
df_Simp = rellenar_filas_vacias_con_media(df)
df_Knimp = rellenar_filas_vacias_con_vecinos(df)

sca = StandardScaler()

df_diff = pd.DataFrame(sca.fit_transform(df_Simp.to_numpy())) -
pd.DataFrame(sca.fit_transform(df_Knimp.to_numpy()))

print(f"{df_diff.abs().sum().sum() / df_diff.size * 100} %")
```

```
0.612737199064422 %
```

2.b) Tratamiento de outliers

Los outliers son los datos de un *dataset* que se alejan considerablemente de la tendencia general y se suelen eliminar porque son datos que pueden causar desajustes en el modelo. En el análisis de datos

también existe la tendencia de dejar los outliers en el entrenamiento porque también se pueden usar para encontrar ciertos patrones no previamente detectados en el *dataset*.

En nuestro caso realizaremos una prueba con y sin quitarlos. Para la función lo que usaremos es aprovechar la matemática estadística que dice que el 99.7% de los datos entran dentro de 3 desviaciones típicas de la media. Por ello lo que hacemos es mantener los datos que tengan menos de 3 en el Z-score (medida que indica cuantas desviaciones típicas está un dato de la media)

```
def quitar_outliers(df):  
    return df[(np.abs(stats.zscore(df)) < 3).all(axis=1)]
```

2.c) Normalización

La normalización es un proceso muy recomendado para el análisis de datos para ajustar las escalas de los valores. En este caso, emplearemos la normalización antes de pasarle los datos al entrenamiento y antes de emplear ciertas funciones que necesitan de datos normalizados.

A lo largo del laboratorio emplearemos dos tipos de normalización diferentes, ambos con el mismo objetivo pero ajustando en escalas ligeramente diferentes y del paquete `sklearn.preprocessing`:

- **StandardScaler**

Con este tipo de normalizador se emplea la siguiente fórmula $x' = \frac{x-u}{s}$. Lo que se realiza es restar el valor entre la media y dividirlo entre la desviación típica para generar una distribución normal.

- **Normalizer**

El Normalizer usa la fórmula L2 Norm. Esta sirve para ajustar los valores entre 0 y 1. Para usar por ejemplo el rango de -1 y 1 se podría usar `MinMaxScaler` por ejemplo.

2.d) SMOTE

El último proceso de preprocesado que vamos a implementar es el SMOTE. El concepto de SMOTE es un algoritmo que sirve para replicar valores en *datasets* desbalanceados. Básicamente el algoritmo sirve para imitar los valores de las clases hasta conseguir que la cantidad de elementos de la clase X e Y sean de un tamaño más similar. Usando esta herramienta podemos obtener valores que nos pueden servir a un mejor entrenamiento. En nuestro caso el *dataset* está ligeramente desbalanceado por lo que se puede aplicar

```
df_smote1 = rellenar_filas_vacias_con_vecinos(df)["diagnosis"] == False  
df_smote2 = rellenar_filas_vacias_con_vecinos(df)["diagnosis"] == True  
print(sum(df_smote1))  
print(sum(df_smote2))
```

```
286  
137
```

Usaremos el siguiente código para aplicar el smote (nótese que para este código ya separamos el X y el Y de los datos)

```
def smote_agregar_samples(x, y):  
    smote = SMOTE(random_state = 25)
```

```
Xnuevo, Ynuevo = smote.fit_resample(x, y)
return Xnuevo, Ynuevo
```

3) Reducción de dimensionalidad

La reducción de dimensionalidad es una técnica que permite reducir la cantidad de dimensiones o variables con la que cuentan nuestros datos. Esto es útil si, por ejemplo, varias de nuestras variables están altamente correlacionadas, ya que datos con menor dimensionalidad son más fáciles de clasificar. Por otro lado, si los datos no estuviesen correlacionados (es decir, si fuesen completamente independientes unos de los otros), obviamente la reducción de dimensionalidad no sería aplicable.

En el caso de nuestro *dataset*, se puede ver en la Figura 2 que hay muchos pares de dimensiones correlacionados positivamente (entre 0.6 y 0.8 de covarianza), y algunos pares de dimensiones que están casi perfectamente correlacionados (más de 0.8 de covarianza).

Los grupos de atributos altamente correlacionados se puede explicar por el hecho de que varias dimensiones están relacionadas con la media y la varianza del radio, parámetro y área de los núcleos celulares (obviamente, todos esos atributos están completamente correlacionados). Por otro lado, las dimensiones relacionadas con la media y la varianza de la compatación y concavidad de los núcleos celulares también están altamente correlacionadas.

Por todo lo visto anteriormente, lo más razonable será aplicar alguna técnica de reducción de dimensionalidad, como PCA o LDA.

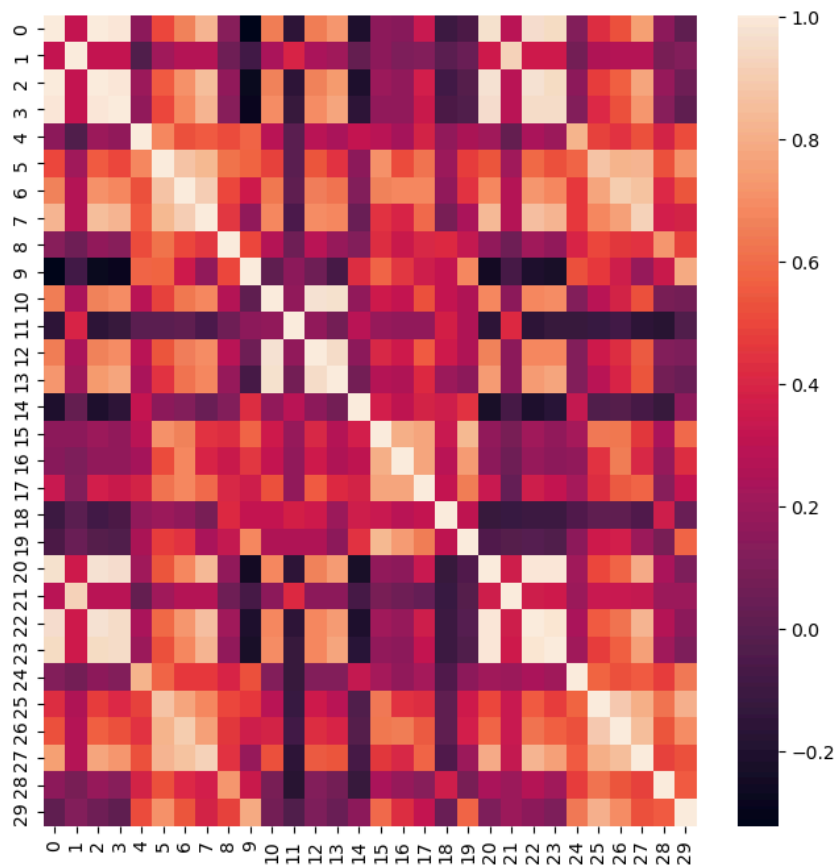


Figura 2: Covarianza entre las 29 dimensiones (normalizadas) del *dataset* de cáncer de mama.

3.a) Principal Component Analysis (PCA)

Este es un método de reducción de dimensionalidad en el que los datos se transforman linealmente a un nuevo sistema de coordenadas, de tal manera que los componentes principales de dicho sistema sean capaces de explicar la mayor cantidad de varianza posible.

Para seleccionar el número apropiado de dimensiones del nuevo sistema de coordenadas podemos utilizar una gráfica para mostrar la varianza explicable según el número de dimensiones utilizadas. Ahí veremos qué dimensiones son capaces de explicar el 90% o 95% de la varianza de los datos.

Como se puede ver en las Figura 3, Figura 4, y Figura 5, la varianza apenas se reduce al eliminar 5 o 10 dimensiones, precisamente debido a que, como vimos en el apartado anterior, muchas dimensiones están altamente correlacionadas entre sí. A partir de menos de 7 a 10 dimensiones la reducción de varianza explicable se vuelve mucho más pronunciada. Por lo que utilizaremos entre 7 y 10 dimensiones para el modelado de nuestros datos.

Es destacable que los 3 métodos distintos utilizados para rellenar datos vacíos no afectan de forma significativa a los resultados.

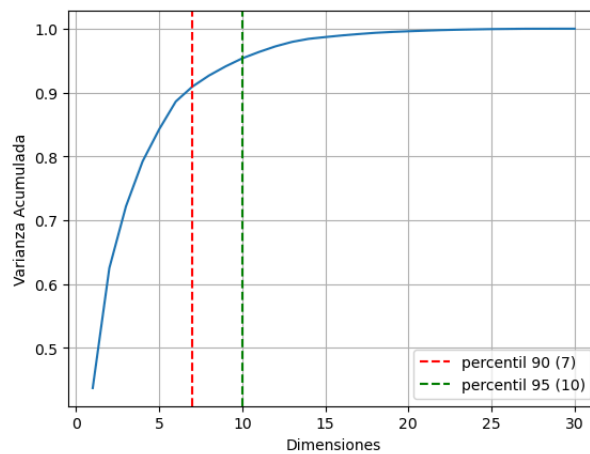


Figura 3: Varianza explicable según el número de dimensiones del PCA (eliminando filas con datos vacíos).

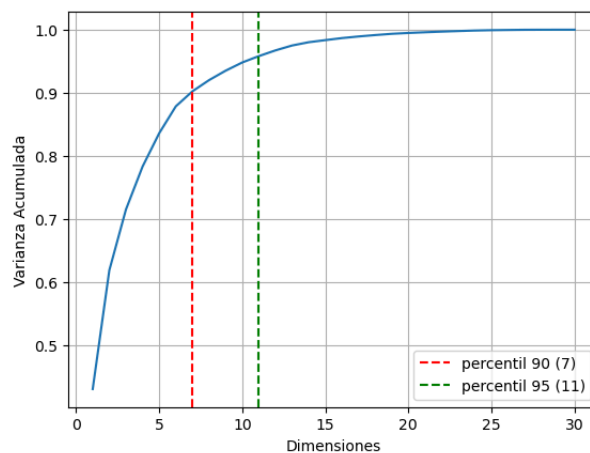


Figura 4: Varianza explicable según el número de dimensiones del PCA (rellenando filas vacías con la media de las columnas).

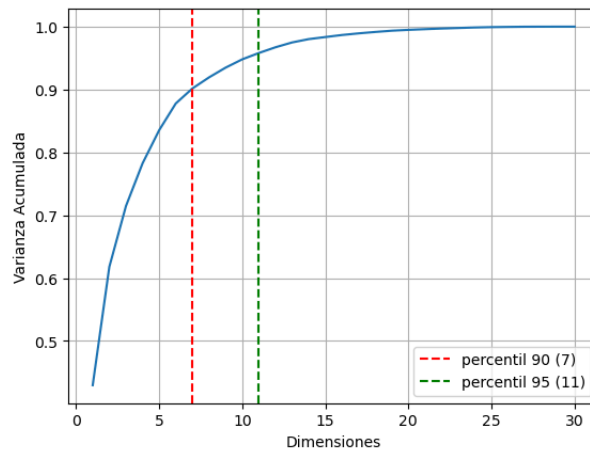


Figura 5: Varianza explicable según el número de dimensiones del PCA (rellenando filas vacías con datos de vecinos más próximos).

3.b) *Linear Discriminant Analysis (LDA)*

Esta técnica de reducción de dimensionalidad permite buscar una combinación lineal de atributos que maximice la separación entre las clases de los datos (es, por tanto, un método supervisado, al contrario que PCA). Además, al contrario que en PCA, el número de dimensiones que reducimos depende del número de clases. En nuestro caso, como solo tenemos dos clases, debemos reducir todas las dimensiones menos una. Por ello este método es menos útil que PCA para entrenar nuestros modelos.

La Figura 6 nos permite mostrar que las clases «maligno» y «benigno» son *casi* linealmente separables, aunque hay un solapamiento significativo entre las dos.

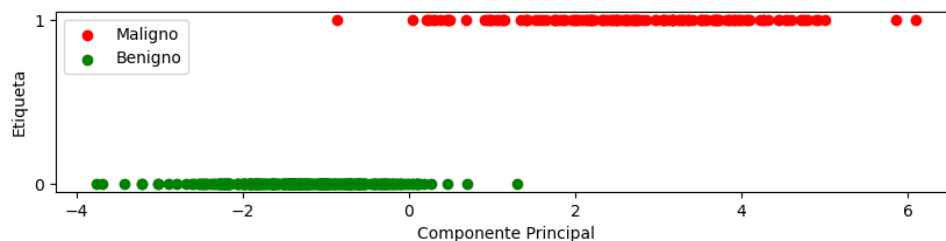


Figura 6: LDA sobre los datos de nuestro *dataset*, separando según las clases «maligno» y «benigno».

4) Creación y Validación

El proceso de creación y validación de modelos es mucho más sencillo de lo que se puede pensar. Esto se debe a que todas las clases de *sklearn* que son modelos de predicción tienen la misma estructura, por lo que realmente el proceso que vamos a realizar es análogo no solo para las clases mencionadas, sino para la gran mayoría de predictores de la librería. De modelo a modelo, la diferencia principal son los hiperparámetros.

El proceso es el siguiente:

1. Cogemos un *dataset*, lo preprocesamos según lo que hayamos establecido (en nuestro caso lo dicho en apartados anteriores) y lo dividimos en Train y Test. (por ejemplo con `sklearn.model_selection.train_test_split`).
2. Escogemos un modelo a validar (digamos el `KNeighborsClassifier` del enunciado) y escogemos sus hiperparámetros (digamos `n_neighbors=n`). Para hacer la función más genérica lo que hacemos es guardar los parámetros todos en un diccionario.

3. Instanciamos el modelo de la siguiente forma:

```
#generico
instancia = modelo(**hiperparametros)
#ejemplo especifico
hip = {n_neighbors=5}
knn = KNeighborsClassifier(**hip)
```

4. Lo siguiente que vamos a hacer es entrenar el modelo para que se ajuste a nuestro *dataset* de entrenamiento que obtuvimos en el paso dos. Esto se hace con la función `fit`, que recibe el *dataset* de entrada y de salida (X e Y) para ajustar el modelo y predecir Y.

```
#generico
instancia.fit(X_train)
#ejemplo especifico
knn.fit(X_train)
```

5. Por último, para poder validar la efectividad, usaremos la función `predict`, que recibe un conjunto X y predice un conjunto Y para determinar la efectividad del modelo. Tenemos el split de test del apartado 2 y simplemente hay que comparar la Y predicha del modelo con la Y verdad del split.

```
#generico
ypred = instancia.predict(X_test)
comparar(ypred, Y_test)
#ejemplo especifico
ypred = knn.predict(X_test)
comparar(ypred, Y_test)
```

Destacar de nuevo, que el proceso es totalmente análogo para el resto de módulos puesto que las funciones usadas, `fit` y `predict` las poseen todos. Simplemente hay que cambiar el nombre de la clase que instanciamos y el diccionario de hiperparámetros a uno válido.

Sin embargo, en los requisitos de la práctica se nos pide utilizar el método de entrenamiento de validación cruzada y entrenar todos los modelos al mismo tiempo. Esto, sorprendentemente, simplifica algunas cosas, puesto que `sklearn` presenta funciones que implementa la validación cruzada, por lo que no vamos a tener que probar a mano cada modelo con cada hiperparámetro. Cabe destacar que `sklearn` usa también `fit` y `predict` también en el `cross_validator` por lo que (aunque se vaya a explicar a continuación) la única diferencia es que el diccionario de hiperparámetros tiene que tener un «arraylike» de `value` (que será un cúmulo de posibles valores del hiperparámetro).

4.a) Validación Cruzada Anidada

Antes de explicar el procedimiento de validación cruzada vamos a presentar los términos y funciones que se van a utilizar durante la explicación del procedimiento en sí

- **Validación cruzada**

Es un modo de validación de modelos. A grandes rasgos lo que hace es probar un modelo con varias combinaciones entrenamiento-test para evaluar como actúa con diferentes splits. (Explicado en mayor manera después)

- **Split**

Consideraremos un split de datos un subconjunto de los datos del *dataset*. Esto quiere decir que por ejemplo un split de [1, 2, 3, 4] es [1, 2]. Por la naturaleza de las funciones a utilizar, los splits no se simbolizan como subconjuntos sino como una lista de índices, luego tu con esos índices obtienes el subconjunto del dataset

- **StratifiedKFold**

`sklearn.model_selection.StratifiedKFold` es la función que vamos a utilizar para generar los splits del *dataset* en la práctica. Esta función es específica para clasificadores con dos clases (benigno, maligno en nuestro caso) y sirve para obtener sets de entrenamiento y de test con una distribución equivalente de dichas clases. A esta función le pasas el número de Splits que quieres generar y puedes usarla de dos maneras, como un generador a través de su función `.split()` o como un argumento de ciertas funciones de `sklearn` a través de «cv»

```
#instanciamos
fuera_cv = StratifiedKFold(n_splits=4)
#generador
for (ixs_train_fuera, ixs_test_fuera) in fuera_cv.split(X, Y):
    hacer_algo()
#argumento
gscv = GridSearchCV(LogisticRegression(), hiperparametros[1], cv=fuera_cv)
```

- **Cross_validate**

Es la clase que `sklearn` pone a disposición de los usuarios para poder realizar la validación cruzada de manera automática. Para que funcione tenemos que pasar una instancia de un modelo, el conjunto X de entrada, el conjunto Y de salida, y luego de manera optativa puede recibir un generador de splits a través de «cv» y podemos establecer que nos devuelva los modelos que entrena a través de «return_estimator=True».

La propia función será la que se encargue de entrenar y testear la instancia pasada como argumento para diferentes splits de los conjuntos de datos pasados. La función nos devuelve un diccionario de datos, entre los cuales destacamos los «test_score» (valores de test) de los modelos para los splits y «estimator» donde encontramos las instancias de los splits

Aquí un ejemplo con el knn que usamos antes

```
#instancia del modelo
hip = {n_neighbors=5}
knn = KNeighborsClassifier(**hip)

#cross_validator con el split de antes
knn_cvs = cross_validate(knn_classifier, X, Y, cv=fuera_cv, return_estimator=True)

#mejor estimador
return knn_cvs["estimator"][np.argmax(knn_cvs["test_score"])]
```

- **GridSearchCV**

Es una función que permite realizar la validación cruzada de varios modelos al mismo tiempo. Funciona de manera similar a `cross_validate`, con la diferencia de que los hiperparametros tienen como «value» un arraylike de valores y que hay que usar `fit` manualmente. De esta manera probamos en

un «grid» cada uno de las combinaciones modelo+hiperparametro+split obteniendo de esta manera resultados bidimensionales en vez de unidimensionales como con `cross_validate`.

Conceptualmente es bastante similar. La función recibe como parámetros el modelo y sus combinaciones de hiperparameters. Luego la instanciamos y realizamos un fit (que realizará tanto el entrenamiento como el testeo) como si tuviesemos un modelo normal (explicado antes) con la diferencia de que de tenemos un grid de resultados

```
#formato de hiperparametros
hiperparametros = {"n_neighbors": (1, 2, 3, 4)}

#instanciamos el modelo
fitted_model_knn = GridSearchCV(KNeighborsClassifier(), hiperparametros,
cv=fuera_cv)

#obtenemos el array de resultados
resultados = fitted_model_knn.cv_results_[mean_test_score]

#obtenemos el indice de la mejor combinación de hiperparametros
best_index = fitted_model_knn.best_index_
```

4.b) Proceso de Validación Completo

Todo lo anterior se junta en una única función que sirve para realizar evaluación cruzada anidada de forma genérica para varios modelos con varios conjuntos de parámetros.

```
def CV1(
    X,
    Y,
    model_params=[],
    metric="f1",
    out_split=3,
    in_split=4,
):
    # Folds del dataset con balanceado de clases.
    fuera_cv = StratifiedKFold(n_splits=out_split, shuffle=True, random_state=25)
    dentro_cv = StratifiedKFold(n_splits=in_split, shuffle=True, random_state=25)

    # Inicializar resultados del bucle exterior.
    results_fuera = {}
    for model,params in model_params:
        results_fuera[model.__name__] = { 'best_index': [] }

    # Ir por cada uno de los folds probando los mismos modelos
    # con todas las combinaciones de parámetros posibles.
    for i, (ixs_train_fuera, ixs_test_fuera) in enumerate(fuera_cv.split(X, Y)):
        # Datos de entrenamiento para el bucle exterior
        X_train_fuera = X[ixs_train_fuera]
        Y_train_fuera = Y[ixs_train_fuera]

        for model,params in model_params:
            # Prueba todas las combinaciones de parámetros posibles.
            fitted_model = GridSearchCV(
                model(),
```

```

        params,
        cv=dentro_cv,
        scoring=metric
    )
    fitted_model.fit(X_train_fuera, Y_train_fuera)

    # Guardamos los parámetros utilizados y el mejor índice.
    results_fuera[model.__name__]['best_index']\
        .append(fitted_model.best_index_)
    results_fuera[model.__name__]['params'] = fitted_model\
        .cv_results_["params"]

# Realizar cross validation final con los mejores
# parámetros de cada modelo.
results = {}
for model,params in model_params:
    # Encontrar el conjunto de parámetros que fue más veces el mejor
    # y seleccionarlos para realizar cross validation.
    index_best = st.mode(results_fuera[model.__name__]['best_index'])
    best_model = model(**results_fuera[model.__name__]['params'][index_best])

    model_csv = cross_validate(
        best_model,
        X,
        Y,
        cv=fuera_cv,
        return_estimator=True,
        scoring=metric
    )

    results[model.__name__] = {
        'best': model_csv["estimator"][np.argmax(model_csv["test_score"])],
        'scores': model_csv["test_score"]
    }

return results

```

5) Resultados

Para la validación cruzada final se ha decidido procesar los datos de la siguiente manera:

```

pca = pca_dataframe(df, n_componentents=7)
x,y = extraer_datos_xy(
    df,
    pre_process=rellenar_filas_vacias_con_vecinos,
    remove_outliers=True,
    smote_augmentation=True,
    standarize=True,
    pca=pca
)

```

Es decir, utilizando reducción de dimensionalidad a 7 componentes, eliminando outliers (hemos comprobado que mejora ligeramente los resultados), con SMOTE para balancear las clases, y estandarización de las entradas.

Adicionalmente, se hacen siempre 3 *fold*s en la validación exterior, y 4 *fold*s en la validación interior de todas las combinaciones de parámetros. Además, se utiliza como métrica el «F1 score».

En cuanto a los parámetros elegidos para la validación, son los siguientes:

```
params = [  
    (KNeighborsClassifier, {  
        "n_neighbors": (1,2,3,6,9,11,14,20,25,30,35,40,45,50),  
    }),  
    (LogisticRegression, {  
        "C": (0.01, 0.0125, 0.025, 0.05, 0.1, 0.15, 1, 10, 40, 50, 60, 100),  
        "penalty": ('none', "l2"),  
        "random_state": (25,) ,  
    }),  
    (DecisionTreeClassifier, {  
        "max_depth": (2,3,4,5,6,8,9,10),  
        "min_samples_split": (2,3,4,5,6,7,8,9,10),  
        "min_samples_leaf": (1,2,3,4,5,6),  
        "random_state": (25,) ,  
    }),  
    (MLPClassifier, {  
        "hidden_layer_sizes": ([3],[4],[5],[6],[7],[3,3],[4,4], [4,4,4], [4,8,4]),  
        "learning_rate_init": (0.001, 0.01, 0.1),  
        "max_iter": (4000,) ,  
        "random_state": (25,) ,  
    })  
]
```

5.a) Hiperparámetros elegidos

- **KNeighborsClassifier:** Este modelo ha sido probado de forma conjunta, y es el más sencillo de probar, pues hay un único hiperparámetro que toma valores discretos. En este caso, se ha decidido probar una gran cantidad de valores, desde pequeños a grandes. El número óptimo de vecinos varía bastante de ejecución a ejecución. La ejecución final con la que nos hemos quedado ha elegido K=45, obteniendo un F1 score de 92-97%
- **LogisticRegression:** se han probado bastantes valores de C de distintas escalas, al igual que se ha probado la regresión logística sin `penalty`. Al igual que KNeighborsClassifier, el número de parámetros óptimos varía bastante de ejecución a ejecución, desde C=0.025, hasta C=50. En la ejecución final con la que nos hemos quedado se ha elegido C=50, obteniendo un F1 score de 94-98%. Es consistentemente el modelo que consigue los mejores resultados.
- **DecisionTreeClassifier:** como en KNeighborsClassifier, los hiperparámetros toman valores discretos, por lo que es fácil probar todas las combinaciones de parámetros razonables que podríamos tener. Para la máxima profundidad se han probado todos los valores menores que 6, y algunos por encima de 6. Para el resto de hiperparámetros, se han probado todos los valores menores que 6 o 10. En la ejecución final con la que nos hemos quedado se ha elegido `max_depth=5`, `min_samples_split=7`, `min_samples_leaf=1`, obteniendo un F1 score de 90-95%. Es consistentemente el peor modelo de los 4.
- **MLPClassifier:** es el modelo más difícil de ajustar debido a que hay muchas combinaciones posibles de número de capas ocultas y tamaños de capas. Se ha decidido probar bastantes tamaños distintos de una sola capa, y un par de tamaños para 2 o 3 capas. No se ha probado con demasiadas neuronas, porque se ha comprobado que los mejores hiperparámetros corresponden con un número de neuro-

nas reducidos. Se ha probado con un número de iteraciones elevado, y dos learning rates distintos, de 0.01 y 0.001. En la ejecución final con la que nos hemos quedado se ha elegido una única capa oculta de 3 neuronas, con un learning rate de 0.001, obteniendo un F1 score de 95-99%. Es el mejor modelo de los 4, muy cerca del regresor logístico.

Los modelos finales conseguidos son los siguientes:

```
{'KNeighborsClassifier':
  {
    'best': KNeighborsClassifier(n_neighbors=45),
    'scores': array([0.94610778, 0.94857143, 0.95454545])
  },
  'LogisticRegression': {
    'best': LogisticRegression(C=50, random_state=25),
    'scores': array([0.94252874, 0.98324022, 0.98265896])
  },
  'DecisionTreeClassifier': {
    'best': DecisionTreeClassifier(
      max_depth=5,
      min_samples_split=7,
      random_state=25
    ),
    'scores': array([0.90909091, 0.93103448, 0.94797688])
  },
  'MLPClassifier': {
    'best': MLPClassifier(hidden_layer_sizes=[3], max_iter=4000, random_state=25),
    'scores': array([0.95402299, 0.99428571, 0.98265896])
  }
}
```

5.b) Tablas

Si bien el resultado de arriba son la combinación de todo el preprocesamiento, también probamos a utilizar simplemente algunas combinaciones de preprocesado (con/sin smote, con/sin outliers, etc)

KneighborsClassifier

	nombre	n_neighbors	preprocesamiento	smote	outliers quitados	mejor score	media
0	KNeighborsClassifier	n_neighbors = 1	quitar	No	No	0.972973	0.96357 +- 0.00822
1	KNeighborsClassifier	n_neighbors = 3	media	No	No	0.917647	0.86684 +- 0.04404
2	KNeighborsClassifier	n_neighbors = 3	knn	No	No	0.919540	0.87206 +- 0.04205
3	KNeighborsClassifier	n_neighbors = 1	knn	No	Si	0.888889	0.87829 +- 0.0094
4	KNeighborsClassifier	n_neighbors = 25	knn	Si	No	0.944444	0.92449 +- 0.01783
5	KNeighborsClassifier	n_neighbors = 45	knn	Si	Si	0.954545	0.94974 +- 0.00434

LogisticRegresion

	nombre	C	penalty	preprocesamiento	smote	outliers quitados	mejor score	media
0	LogisticRegression	C = 0.01	penalty = l2	quitar	No	No	0.958333	0.95344 +- 0.00847
1	LogisticRegression	C = 10	penalty = l2	media	No	No	0.978261	0.93666 +- 0.03711
2	LogisticRegression	C = 10	penalty = l2	knn	No	No	0.978261	0.93666 +- 0.03711
3	LogisticRegression	C = 10	penalty = l2	knn	No	Si	0.953846	0.92788 +- 0.0227
4	LogisticRegression	C = 10	penalty = l2	knn	Si	No	0.984456	0.97354 +- 0.01457
5	LogisticRegression	C = 40	penalty = l2	knn	Si	Si	0.983240	0.96948 +- 0.02334

DecisionTreeClassifier

	nombre	max_depth	min_samples_split	min_samples_leaf	preprocesamiento	smote	outliers quitados	mejor score	media
0	DecisionTreeClassifier	max_depth = 5	min_samples_split = 4	min_samples_leaf = 1	quitar	No	No	0.938776	0.9348 + 0.00546
1	DecisionTreeClassifier	max_depth = 3	min_samples_split = 2	min_samples_leaf = 1	media	No	No	0.923077	0.89406 + 0.03171
2	DecisionTreeClassifier	max_depth = 2	min_samples_split = 2	min_samples_leaf = 5	knn	No	No	0.883721	0.86141 + 0.02051
3	DecisionTreeClassifier	max_depth = 3	min_samples_split = 2	min_samples_leaf = 4	knn	No	Si	0.892308	0.86795 + 0.026
4	DecisionTreeClassifier	max_depth = 3	min_samples_split = 2	min_samples_leaf = 6	knn	Si	No	0.929730	0.92445 + 0.00756
5	DecisionTreeClassifier	max_depth = 5	min_samples_split = 7	min_samples_leaf = 1	knn	Si	Si	0.947977	0.92937 + 0.0195

MLPClassifier

	nombre	hidden_layer_sizes	learning_rate_init	max_iter	preprocesamiento	smote	outliers quitados	mejor score	media
0	MLPClassifier	hidden_layer_sizes = [4, 4, 4]	learning_rate_init = 0.1	max_iter = 4000	quitar	No	No	0.986111	0.9729 + 0.01306
1	MLPClassifier	hidden_layer_sizes = [7]	learning_rate_init = 0.001	max_iter = 4000	media	No	No	1.000000	0.9381 + 0.05967
2	MLPClassifier	hidden_layer_sizes = [7]	learning_rate_init = 0.001	max_iter = 4000	knn	No	No	1.000000	0.93468 + 0.06037
3	MLPClassifier	hidden_layer_sizes = [7]	learning_rate_init = 0.01	max_iter = 4000	knn	No	Si	0.939394	0.92847 + 0.00975
4	MLPClassifier	hidden_layer_sizes = [5]	learning_rate_init = 0.001	max_iter = 4000	knn	Si	No	0.989583	0.98054 + 0.01557
5	MLPClassifier	hidden_layer_sizes = [3]	learning_rate_init = 0.001	max_iter = 4000	knn	Si	Si	0.994286	0.97699 + 0.02072

Vemos como en general los resultados entre el MLP, la regresión logística y el kNeighborsClassifier son muy buenos, y los del árbol son algo peores. A mayores vemos que la combinación de todos los elementos del preprocesamiento da los mejores resultados

5.c) Pruebas en *dataset* de test

Los 4 modelos anteriores se han probado en un pequeño *dataset* de test con 4 ejemplos.

Modelo	Ejemplo 1	Ejemplo 2	Ejemplo 3	Ejemplo 4
KNeighborsClassifier	0	0	0	1
LogisticRegression	0	0	1	1
DecisionTreeClassifier	0	0	0	1
MLPClassifier	0	0	1	1
	0	0	1	1

Tabla 1: Resultados de los 4 modelos para el *dataset* de test.

Se puede ver que el regresor logístico y el MLP consiguen las predicciones correctas. Por otro lado, los otros 2 modelos tienen un falso negativo cada uno.

6) Bibliografía

<https://www.turing.com/kb/smote-for-an-imbalanced-dataset>

https://en.wikipedia.org/wiki/Linear_discriminant_analysis

https://en.wikipedia.org/wiki/Logistic_regression

https://en.wikipedia.org/wiki/Decision_tree_learning